

DA3452 – Speech and Language Technology

Weekend Reading and Coding Assignment 1

Words, Unicode, UTF-32, and UTF-8

Status of this assignment. This sheet is **not for submission**. However, every student is responsible for the reading, concepts, hand calculations, and coding exercises described here. The material may be tested in the module assessment and the end-semester examination.

By the end of this work, you should be able to:

- distinguish words, morphemes, characters, Unicode code points, and bytes;
- explain why Unicode and UTF-8 are not the same thing;
- manually convert selected Unicode code points into UTF-8 byte sequences;
- implement simple UTF-32 big-endian and UTF-8 encoders; and
- verify your implementation against Python’s standard library.

Part A: Required reading

Read the following portions of Daniel Jurafsky and James H. Martin, *Speech and Language Processing*, 3rd edition draft, Chapter 2:

- **Section 2.1 – Words**
- **Section 2.2 – Morphemes: Parts of Words**
- **Section 2.3 – Unicode**, including
 - Section 2.3.1 – Code Points
 - Section 2.3.2 – UTF-8 Encoding

You are not required to read Section 2.4 on Byte-Pair Encoding for this assignment.

Questions you should be able to answer

Prepare brief answers, in your own words, to the following questions:

1. Why is whitespace not a universal method for identifying words?
2. What is the difference between a morpheme and a character?
3. What is a Unicode code point?
4. What is the difference between an abstract character and a glyph?
5. What is the difference between Unicode and UTF-8?
6. Why is storing every code point using four bytes wasteful for ordinary English text?
7. Why is every ASCII byte sequence also a valid UTF-8 byte sequence?
8. Does one visible character always correspond to one code point? Give an example.

Part B: Manual UTF-8 conversion

Choose **two** of the following non-ASCII characters. At least one choice must require three or four UTF-8 bytes.

Character	Description	Code point
é	Latin small letter e with acute	U+00E9
अ	Devanagari letter A	U+0905
த	Tamil letter TA	U+0BA4
😊	Smiling face with smiling eyes emoji	U+1F60A

For each chosen character:

1. Write its Unicode code point in hexadecimal.
2. Convert the code point to binary.
3. Determine whether UTF-8 uses one, two, three, or four bytes for it.
4. Insert the code-point bits into the appropriate UTF-8 bit pattern.
5. Write the final UTF-8 bytes in binary and hexadecimal.
6. Verify the answer using Python.

For example, your verification code may look like this:

```
character = "\u0905" # DEVANAGARI LETTER A

print("Character:", character)
print("Code point:", f"U+{ord(character):04X}")
print("UTF-8 bytes:", character.encode("utf-8"))
print(
    "UTF-8 hexadecimal:",
    [f"{byte:02X}" for byte in character.encode("utf-8")]
)
print(
    "UTF-8 binary:",
    [f"{byte:08b}" for byte in character.encode("utf-8")]
)
```

A useful check is:

U+0905	→	E0 A4 85
--------	---	----------

Part C: Implement direct four-byte encoding

Implement a function that encodes one valid Unicode code point using exactly four bytes in **big-endian** order. Do not call `str.encode()` inside your implementation.

```
def encode_utf32be_codepoint(code_point: int) -> bytes:
    """
    Encode one valid Unicode code point as four bytes
    in big-endian order.
    """
    pass
```

Your function should produce results of the following form:

U+0068	00 00 00 68
U+0905	00 00 09 05
U+1F60A	00 01 F6 0A

Next, encode an entire Python string by applying the function to each code point:

```
def encode_utf32be(text: str) -> bytes:
    """Encode an entire Python string in UTF-32 big-endian."""
    pass
```

Verify your result against Python:

```
assert encode_utf32be(text) == text.encode("utf-32-be")
```

Use `utf-32-be` for verification, not simply `utf-32`. The latter may include a byte-order mark, which is outside the scope of this exercise.

Part D: Implement UTF-8 encoding

Implement UTF-8 encoding for one valid Unicode code point. Use bit shifts, masks, and construction of a bytes object. Do not call `str.encode()` inside your implementation.

```
def encode_utf8_codepoint(code_point: int) -> bytes:
    """
    Encode one valid Unicode code point using UTF-8.
    Handle the one-, two-, three-, and four-byte cases.
    """
    pass
```

Your implementation should use the following ranges:

Code-point range	UTF-8 length
U+0000 - U+007F	1 byte
U+0080 - U+07FF	2 bytes
U+0800 - U+FFFF	3 bytes
U+10000 - U+10FFFF	4 bytes

Then encode a complete string:

```
def encode_utf8(text: str) -> bytes:
    """Encode an entire Python string using encode_utf8_codepoint()."""
    pass
```

Part E: Test and compare

Test both encoders on at least the following strings:

```
test_strings = [
    "hello",
    "Caf\u00E9",
    "\u0905",
    "\u0BA4\u0BAE\u0BBF\u0BB4\u0BCD",
    "Hello \u0001F60A",
]
```

Use automated checks:

```
for text in test_strings:
    assert encode_utf8(text) == text.encode("utf-8")
    assert encode_utf32be(text) == text.encode("utf-32-be")
```

```
print("All tests passed.")
```

Create, for your own study, a table with the following columns:

Text	Code points	UTF-8 byte count	UTF-32 byte count
------	-------------	------------------	-------------------

Questions you should be able to answer after coding

1. Why does UTF-32 require four bytes per code point?
2. Why does UTF-8 save space for `hello`?
3. For which of the test strings is the UTF-8 saving over UTF-32 smallest?
4. Does Python's `len(text)` return the number of UTF-8 bytes? Explain.
5. Why is checking against the standard library valuable even when your own algorithm appears to work on a few examples?

Expected scope. You are responsible for encoding valid Unicode text. You are **not** expected at this stage to implement UTF-8 decoding, malformed-input handling, byte-order marks, or Unicode normalization.

Self-check before the module test

You should be able to complete all of the following without consulting this sheet:

- explain the sequence *abstract character* $\rightarrow$ *code point* $\rightarrow$ *encoded bytes*;
- distinguish UTF-32's fixed-width representation from UTF-8's variable-length representation;
- determine the number of UTF-8 bytes required by a given code-point range;
- manually encode a simple two- or three-byte example;
- explain the output of `ord()`, `encode("utf-8")`, and hexadecimal byte formatting; and
- describe how you validated your encoder against Python.

Keep your notebook and calculations for revision. The work is not collected, but the underlying concepts and procedures are examinable.